



Relatório de projeto

Laboratório de Desenvolvimento de Software

Licenciatura em Engenharia Informática

Aplicação demonstradora da API para geração de
gráficos através de um .csv em Python:

CSVExporter

Equipa:

- Andreia Romão Func. N.º 1702430
- Cátia Santos - Func. N.º 1702194
- Júlio Martins - Func. N.º 1900881
- Mariana Barrote - Func. N.º 2200640
- Miguel Lopes - Func. N.º 1900798

2025

Endereço do repositório Git:

Este relatório apresenta o projeto da aplicação demonstradora *CSVExporter*, desenvolvida pela equipa, composta pelos funcionários Andreia, Cátia, Júlio, Mariana e Miguel e está disponível no repositório *github*, com o seguinte endereço:

https://github.com/catiafsantos/UAb_LDS_Grupo10_ChalkPowder

Descrição geral do projeto

O projeto insere-se no âmbito do estágio na empresa *SimProgramming* e propõe-se a demonstrar como desenvolver uma aplicação utilizando as bibliotecas/APIs *Seaborn*, *Pandas* e *Tkinter* segundo o estilo arquitetónico MVC (*Model-View-Controller*).

Neste projeto optou-se por desenvolver uma aplicação cujo conceito-base é a construção e exportação de gráficos a partir de ficheiros .csv fornecidos pelo utilizador. A aplicação permite a leitura e validação do ficheiro, seleção do tipo de gráfico (linhas ou barras), escolha dinâmica das variáveis a apresentar nos eixos, bem como a personalização dos nomes das variáveis. No final é possível exportar o gráfico resultante para uma diretoria à escolha.

Para promover a coesão interna e a separação de interesses, foram utilizados os princípios do estilo arquitetónico MVC. Para tal, recorreremos ao uso de eventos personalizados que mimicom a lógica de eventos e delegados, garantindo um acoplamento fraco entre os componentes e permitindo maior escalabilidade. Demos também prioridade ao tratamento personalizado de exceções, implementando mensagens claras e respostas adequadas aos erros, orientadas para o utilizador. Por fim, aplicámos o conceito de interfaces (através de funções e estruturas de abstração), nomeadamente na *View* e no *Logger*, de forma a manter a independência entre os elementos da aplicação, facilitando os testes e a manutenção ao longo do tempo.

Neste projeto, o componente *Model* é responsável por importar, validar e processar os dados .csv, assim como por gerar e armazenar gráficos, emitir eventos sobre o estado da aplicação e reportar eventuais problemas que ocorram nesse processo. O componente *View* tem como função construir e apresentar a interface gráfica, desenhar os gráficos e permitir a interação com o utilizador através de botões, menus e campos de texto. Já o componente *Controller* atua como coordenador entre a *View* e o *Model*, orquestrando os eventos da aplicação com base nas ações do utilizador, sendo também o ponto de entrada principal da aplicação.

Foram utilizadas as seguintes bibliotecas/APIs externas:

- A biblioteca/API de geração de gráficos Seaborn
<https://seaborn.pydata.org/>
- A biblioteca/API Pandas para a estrutura de dados
<https://pandas.pydata.org/>
- A biblioteca/API Tkinter para a UI
<https://docs.python.org/3/library/tkinter.html>

No final deste documento, em anexo, existem demonstrações visuais da aplicação.

Funcionamento esquemático

Para o desenvolvimento desta aplicação, adotámos a variante MVC proposta por **Curry e Grace**, na qual os inputs do utilizador são processados na *View* e não prevê comunicação do *Model* para o *Controller*. Esta abordagem permite uma separação mais clara de responsabilidades, promovendo a modularidade e a testabilidade. O Diagrama 1 ilustra, de forma genérica, as responsabilidades de cada componente, bem como as mensagens trocadas entre eles.

Relatório de projeto – 2025

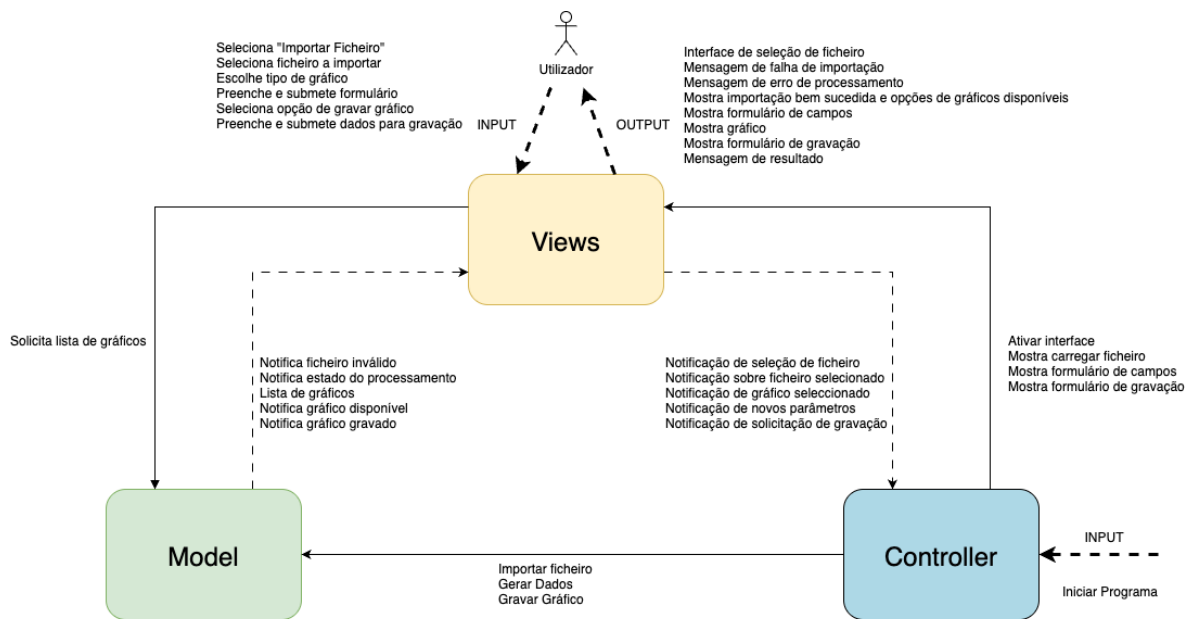


Diagrama 1

O ciclo de execução da aplicação tem início no *Controller*, que estabelece as ligações entre os componentes *Model* e *View*. No entanto, como a interação do utilizador é captada inicialmente pela *View*, o controlo transita de imediato para este componente, que processa os eventos através da interface gráfica implementada em *Tkinter*.

Após a inicialização da aplicação, incluindo a associação de eventos e delegados, o *Controller* retoma o controlo sempre que um evento é reportado pela *View*, analisando a intenção do utilizador e encaminhando comandos ao *Model*. Esta coordenação garante que as decisões relativas ao fluxo de execução, como a geração ou gravação de gráficos, sejam tomadas centralmente, mantendo a aplicação modular, coesa e extensível, em conformidade com os princípios do estilo arquitetónico MVC.

O fluxo geral de execução é apresentado no diagrama de sequência que se segue (Diagrama 2).

Divisão de responsabilidades pelos componentes

Durante o desenvolvimento da nossa aplicação, uma das etapas fundamentais foi a distribuição das responsabilidades pelos componentes do estilo arquitetónico MVC, de acordo com a variante proposta por Curry & Grace (ver Figura 1).

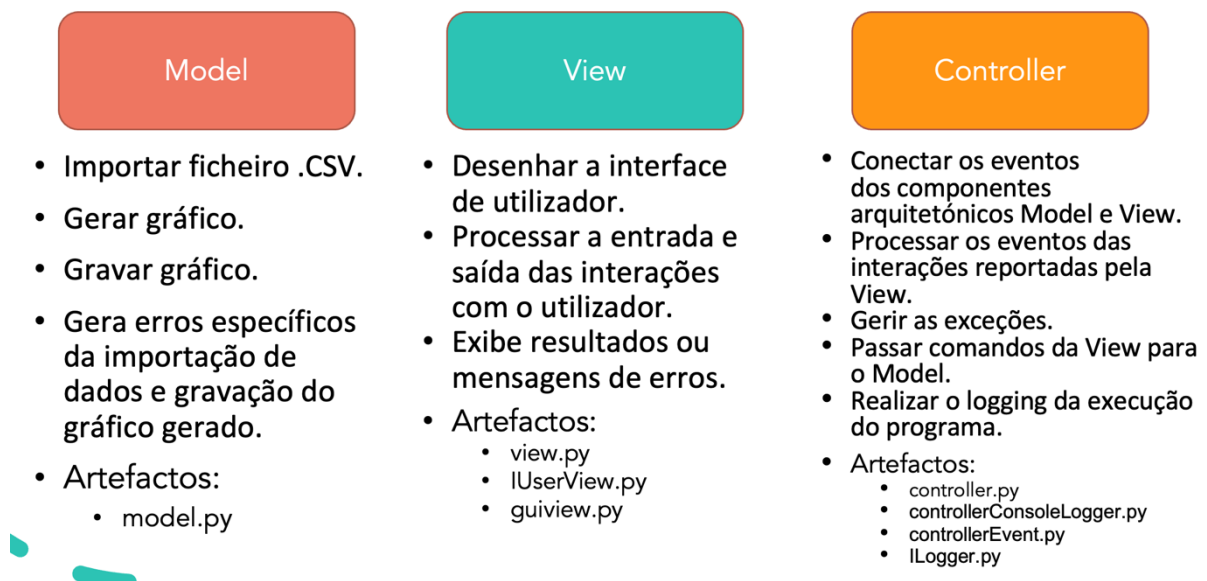


Figura 1

Recurso a eventos para acoplamento fraco

Com o objetivo de enfraquecer o acoplamento entre componentes, implementámos um sistema de eventos e delegados para permitir comunicação entre *Model*, *View* e *Controller* de forma desacoplada. Como esta aplicação foi construída em *Python*, foi necessário implementar uma lógica personalizada, recorrendo a uma classe chamada *ControllerEvent*, de modo a mimicar o comportamento da linguagem *C#*. A Tabela 1 descreve estas ligações, indicando para cada evento quem o emite e quem o consome.

Tabela 1: Componentes emissores e recetores de eventos.

	Model Emissor (ME)	View Emissor (VE)	Controller Emissor (CE)
Model Recetor (MR)		(VE) grava_grafico_click_evt (MR) gravar_grafico (VE) importar_ficheiro_evt (MR) importar_ficheiro	(CE) grava_grafico_evt (MR) gravar_grafico (CE) importar_ficheiro_evt (MR) importar_ficheiro
View Recetor (VR)	(ME) estado_processamento_evt (VR) mostra_mensagem_info (ME) ficheiro_invalido_evt (VR) mostra_erro_ficheiro (ME) grafico_disponivel_evt (VR) atualiza_lista_graficos (ME) grafico_gerado_evt (VR) mostrar_grafico (ME) falha_importacao_evt (VR) mostra_erro_importacao		(CE) mostra_dlg_carregar_ficheiro_evt (VR) mostra_dlg_carregar_ficheiro (CE) mostra_dlg_grava_grafico_evt (VR) mostra_dlg_grava_grafico
Controller Recetor (CR)		(VE) importar_ficheiro_click_evt (CR) user_importa_ficheiro (VE) ficheiro_selecionado_evt (CR) user_seleciona_ficheiro (VE) grafico_selecionado_click_evt (CR) user_selecionou_grafico (VE) submissao_parametros_evt (CR) user_submeteu_parametros (VE) solicita_guardar_grafico_click_evt (CR) user_solicitou_gravacao (VE) grava_grafico_click_evt (CR) user_grava_grafico	

Emprego de exceções respeitando o estilo MVC

No contexto do estilo arquitetónico MVC (*Model-View-Controller*), o tratamento de exceções assume um papel central na robustez e qualidade da experiência do utilizador. É importante que os erros sejam detetados no momento certo, mas geridos de forma centralizada, promovendo a modularidade, a rastreabilidade e a flexibilidade na reação a falhas. Assim, enquanto o *Model* - sendo o componente que realiza mais validações suscetíveis de gerar exceções - se responsabiliza por executar as ações e detetar possíveis erros, o *Controller* coordena a resposta às situações encontradas, decidindo como e quando informar o utilizador, registar *logs* e, especialmente, quando interromper o ciclo de execução. A *View* limita-se a apresentar mensagens claras e orientadas ao utilizador, mantendo a interface responsiva e estável mesmo perante falhas inesperadas.

Neste projeto, foram identificados e tratados diversos tipos de exceções funcionais e técnicas, cada uma refletida em eventos específicos emitidos pelos componentes e acompanhadas de mecanismos de *logging* e, nalguns casos, de feedback visual, conforme descrito nas secções seguintes.

Exceção: Ficheiro inválido

A maioria das situações de exceção consideradas nesta aplicação ocorre quando o utilizador seleciona um ficheiro inválido, por exemplo, que não esteja no formato .csv, não contenha dados ou cujo tamanho excede o limite permitido(10MB), visto que, todas estas situações representam uma falha funcional no início do ciclo de utilização da aplicação. Quando detetadas, o *Model* emite um evento que dará origem a uma mensagem de erro clara para o utilizador (Diagrama 3).

Este tipo de tratamento permite ao utilizador corrigir a situação de forma autónoma, sem impacto estrutural na aplicação.

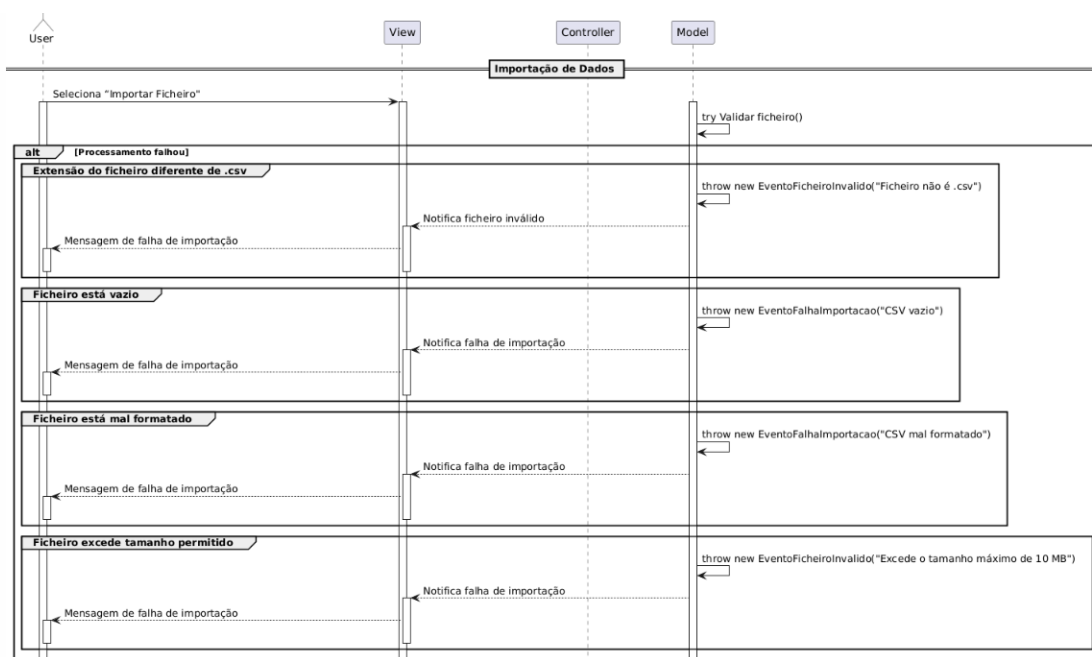


Diagrama 3 – Excerto do diagrama de sequência

Exceção: Falha de geração do gráfico

Outra situação prevista é a falha na geração do gráfico. Caso ocorram problemas na validação ou persistência dos dados, se o ficheiro CSV não estiver disponível ou o tipo de gráfico solicitado não for suportado, verifica-se uma desconformidade entre o fluxo e as capacidades internas da aplicação. Esta exceção é tratada no *Model*, que emite um evento com uma notificação apropriada, garantindo uma abordagem que permite isolar a falha e manter a estabilidade e continuidade da aplicação. No futuro, será possível gerar mensagens de erro específicas para cada situação, caso se pretenda (Diagrama 4).

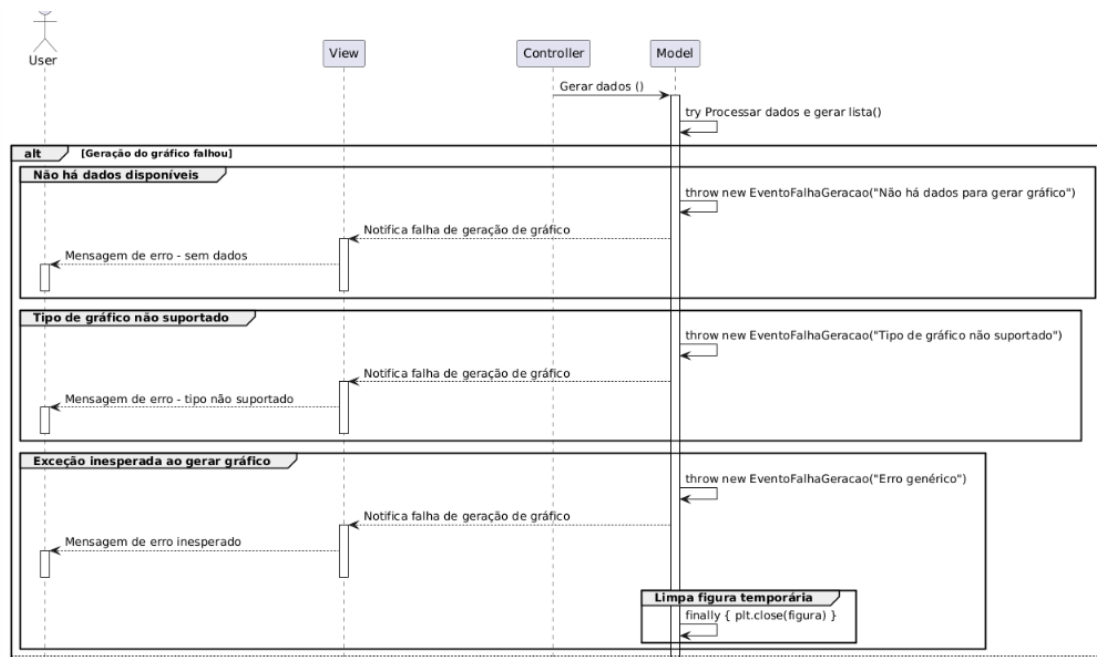


Diagrama 4 - Excerto do diagrama de sequência

Exceção: Erro interno

Por fim, o sistema prevê a possibilidade de ocorrerem erros inesperados em tempo de execução, como exceções *Python* não controladas diretamente pela lógica funcional. Nestes casos, o componente em questão, emite um evento de erro interno, transportando a stack trace do erro, com o objetivo de ser registado via *logger* para posterior análise e possíveis melhorias de aplicação. Este mecanismo permite isolar falhas técnicas sem afetar a robustez geral da aplicação e apanhar as restantes exceções, não mapeadas.

Uso de interfaces para independência entre componentes

A interface *ILogger* foi implementada com o objetivo de abstrair o mecanismo de registo de *logs* entre componentes (código 1), visando garantir baixo acoplamento. Esta interface declara dois métodos: *log_info* e *log_erro*, que permitem registar, respetivamente, mensagens informativas e mensagens de erro. Desta forma, componentes como o *Controller*, por exemplo, podem registar mensagens de log sem necessitar de conhecer os detalhes da sua apresentação ou persistência, garantindo também que, caso seja necessário alterar algum detalhe, não haja impacto em componentes como o *Model* ou a *View*. Na versão atual, a implementação escreve as mensagens de log no terminal durante o tempo de execução, mas poderíamos, por exemplo, modificar o código para que os *logs* fossem enviados para um ficheiro local ou até apresentados via UI para administradores, sem necessidade de modificar os nossos componentes. assegurando flexibilidade, extensibilidade e conformidade com o princípio da inversão de dependência do paradigma de desenvolvimento orientado por interfaces.

```
class ILogger(ABC):
    @abstractmethod
    def log_info(self, mensagem: str) -> None:
        pass

    @abstractmethod
    def log_erro(self, mensagem: str) -> None:
        pass
```

Código 1

Para além do *logger*, foi ainda implementada a interface *IUIView* com o objetivo de abstrair a componente de interface gráfica da aplicação, promovendo uma separação clara entre a lógica de apresentação e o restante núcleo da aplicação (código 2). Esta interface permite ao *Controller* interagir com a interface do utilizador sem depender dos detalhes da sua implementação concreta. Assim, caso seja necessário substituir o *Tkinter* por uma nova versão ou algo diferente, não será necessário alterar

o código do *Controller*, apenas o da *View* de forma a acomodar as mudanças na implementação dos métodos.

```
class IUserView(ABC):

    # Propriedades para eventos
    @property
    @abstractmethod
    def importar_ficheiro_click_evt(self): pass

    @property
    @abstractmethod
    def ficheiro_selecionado_evt(self): pass

    @property
    @abstractmethod
    def grafico_selecionado_click_evt(self): pass

    @property
    @abstractmethod
    def submissao_parametros_evt(self): pass

    @property
    @abstractmethod
    def solicita_guardar_grafico_click_evt(self): pass

    @property
    @abstractmethod
    def grava_grafico_click_evt(self): pass

    # Métodos públicos obrigatórios
    @abstractmethod
    def ativar_interface(self) -> None: pass

    @abstractmethod
    def mostra_dlg_carregar_ficheiro(self) -> None: pass

    @abstractmethod
    def mostra_dlg_grava_grafico(self) -> None: pass

    @abstractmethod
    def mostra_formulario_parametros(self, colunas: List[str]) -> None: pass

    @abstractmethod
    def mostra_mensagem_info(self, mensagem: str) -> None: pass
```

```
@abstractmethod
def atualiza_lista_graficos(self, graficos: List[str]) -> None: pass

@abstractmethod
def mostrar_grafico(self, lista_graficos: List[str]) -> None: pass

@abstractmethod
def mostra_erro_importacao(self, mensagem: str) -> None: pass

@abstractmethod
def mostra_erro_ficheiro(self, mensagem: str) -> None: pass

@abstractmethod
def voltar_menu_inicial(self) -> None: pass
```

Reflexão final

A construção desta aplicação representou um desafio técnico e conceptual significativo, não tanto pela complexidade das funcionalidades em si, mas pela necessidade de estruturar tudo segundo o estilo arquitetónico MVC, promovendo separação de interesses, baixo acoplamento e responsabilidades bem definidas.

No início do projeto, optámos por seguir desde logo a variante do padrão MVC proposta por *Curry & Grace* cuja abordagem nos pareceu mais adequada à arquitetura que pretendíamos implementar. No entanto, a meio do projeto, fomos desafiados a refletir sobre como seria aplicar a variante de *Krasner & Pope*. Embora ainda não existisse código implementado nessa fase, a elaboração dos diagramas segundo essa outra visão revelou-se particularmente complexa. Esse momento obrigou-nos a pensar profundamente sobre as implicações das diferentes interpretações e acabou por reforçar a nossa confiança na escolha feita inicialmente.

Trabalhar em equipa sobre esta arquitetura foi também particularmente desafiante, pois cada elemento ficou com responsabilidades distintas — *Model*, *View*, *Controller*, Testes, Arquitetura e Planeamento -, sem visibilidade direta ou controlo sobre o que os colegas estavam a desenvolver. Esta separação, embora benéfica e

enriquecedora do ponto de vista técnico e académico, exigiu um esforço acrescido de comunicação e colaboração, especialmente porque todos temos vidas ocupadas e horários incompatíveis. Ainda assim, conseguimos alinhar-nos em torno de um objetivo comum, o que nos permitiu chegar ao que entendemos ser uma solução coesa e funcional.

O aspeto mais difícil de todo o projeto foi, sem dúvida, manter o *Model* totalmente desligado da *View*, ao mesmo tempo, que todas as informações necessárias aos fluxos - como estados, sucessos e falhas - fossem comunicadas de forma adequada. O sistema de eventos e delegados, que implementámos manualmente em *Python* através da classe *ControllerEvent*, foi fundamental para este propósito. No entanto, esta abordagem exigiu uma disciplina rigorosa e constantes revisões da arquitetura, especialmente porque *Python* não possui um sistema nativo de eventos delegados, o que levou a decisões estruturais importantes e a mudanças ao longo do desenvolvimento.

Um dos momentos mais enriquecedores do projeto foi o desenho dos eventos de falha. Criar eventos distintos para tratar diversas situações funcionais, bem como exceções internas inesperadas, trouxe não só robustez ao sistema, mas também um sentimento de confiança técnica: construímos algo que não se quebra ao primeiro erro e que responde bem a situações fora do comum.

Em resumo, mais do que construir uma aplicação funcional, esta experiência permitiu-nos experimentar e compreender na prática o valor dos princípios de modularidade, abstração, baixo acoplamento e resiliência a erros. É esse conhecimento que levamos connosco, mais até do que a aplicação em si.

Anexos

Em anexo, alguns printscreens da execução da aplicação.

Disponibiliza um ecrã de Boas-Vindas que permite carregar um ficheiro .csv de uma diretoria local (Figura 2).

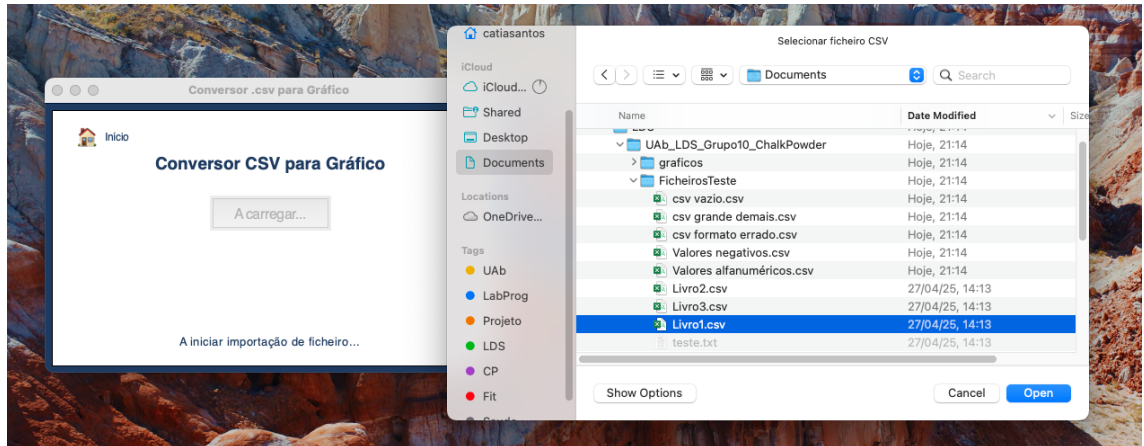


Figura 2

O ficheiro é validado pela aplicação, enviando uma mensagem de erro explícita em caso de falha (lista de erros mais abaixo). No caso de uma validação com sucesso, o utilizador passa então para a escolha do tipo de gráfico a apresentar (Figura 3).

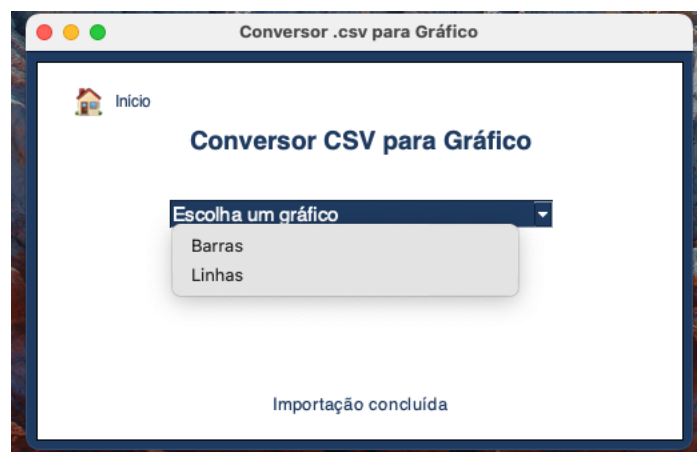


Figura 3

Após selecionar o tipo de gráfico, o utilizador pode escolher qual variável irá ficar em cada eixo, e ainda personalizar os nomes apresentados nos eixos e submeter. (Figura 4).

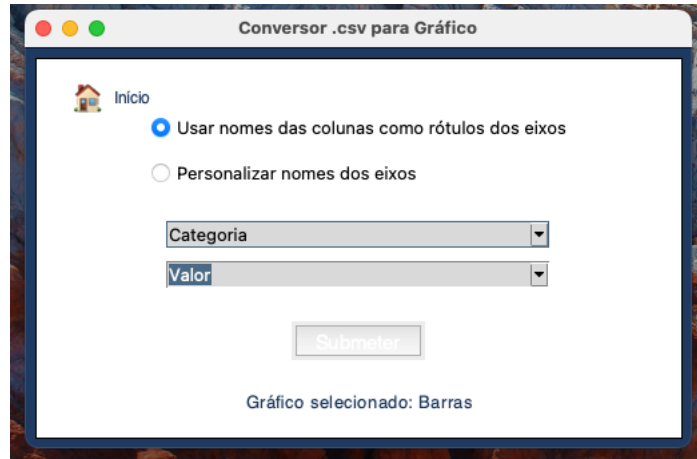


Figura 4

Os dados submetidos vão ser representados no tipo de gráfico escolhido, neste caso, barras: (Figura 5)

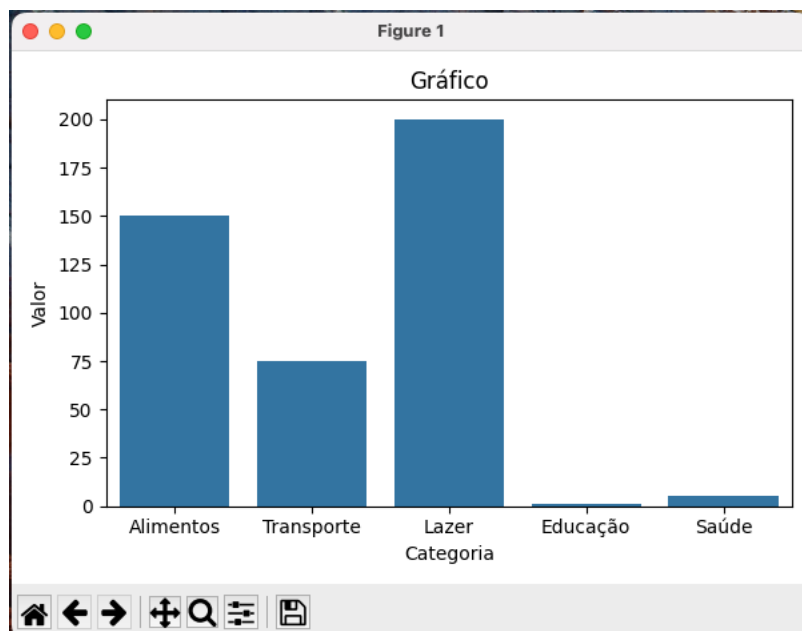


Figura 5

Por fim, o utilizador, para além de poder navegar no gráfico, pode exportá-lo para uma diretoria à sua escolha (Figura 6).

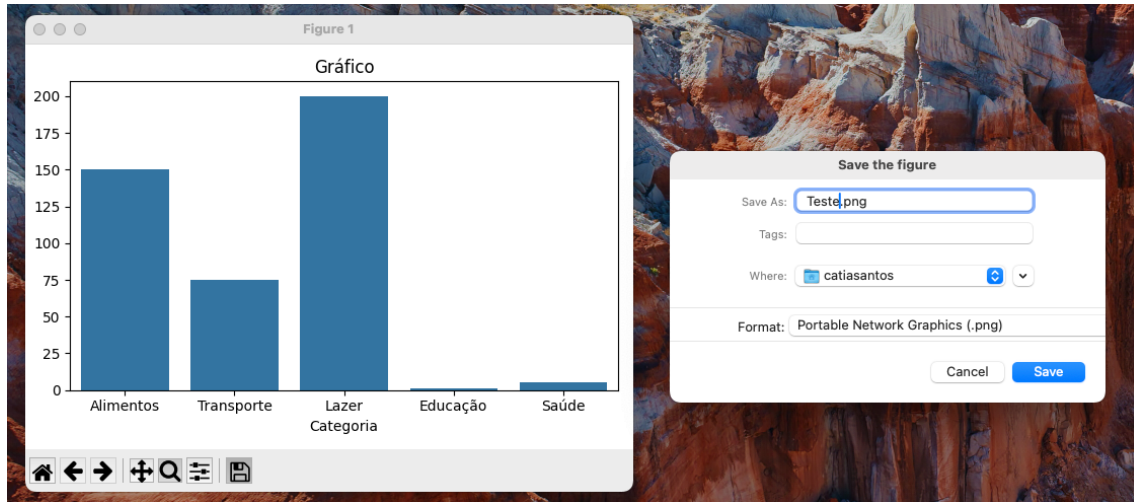
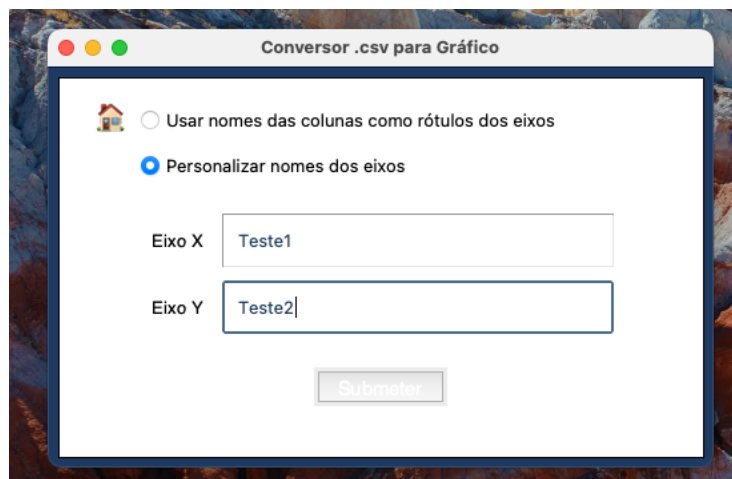


Figura 6

Adicionalmente, podemos ver um exemplo que demonstra a personalização dos nomes das variáveis (Figura 7).



☐ Usar nomes das colunas como rótulos dos eixos

☒ Personalizar nomes dos eixos

Eixo X:

Eixo Y:

Figura 7

E um gráfico de linhas para os mesmos dados previamente utilizados para gerar o gráfico de barras (Figura 8).

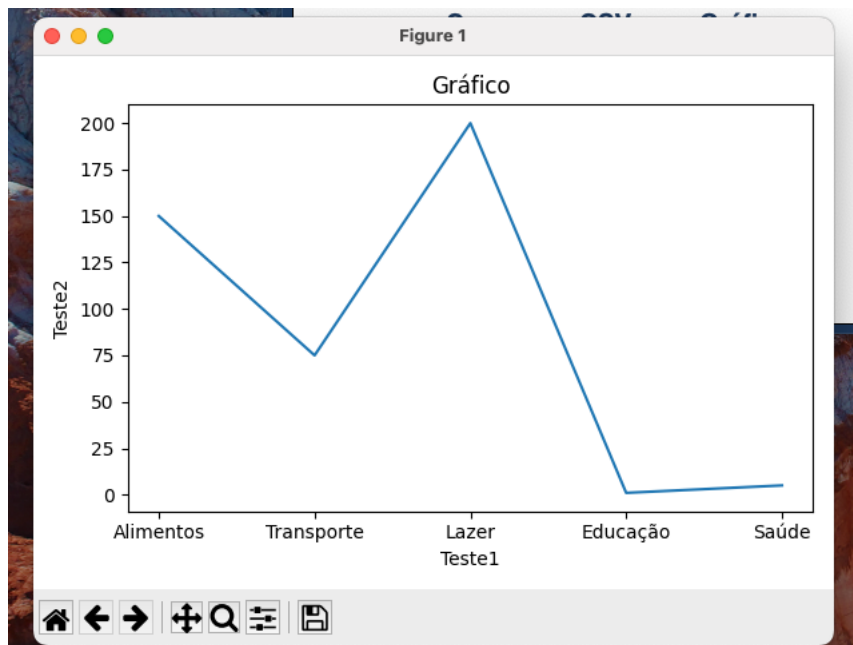


Figura 8

Exemplos de comportamento em caso de erro

Quando o ficheiro está vazio (Figura 9)



Figura 9

Quando o ficheiro está mal formato (Figura 10)



Figura 10

Quando o ficheiro excede o valor recomendado(10MB) (Figura 11)



Figura 11

Exemplo de logs de execução em terminal:

Relatório de projeto – 2025

```
(graficos) catiasantos@MacBook-Pro-de-Catia graficos % python run.py
[INFO] A iniciar a interface gráfica...
[INFO] ativar_interface() - Interface gráfica iniciada.
[INFO] on_importar_ficheiro_click() - Botão 'Importar Ficheiro' clicado.
[INFO] mostra_dlg_carregar_ficheiro() - Ficheiro selecionado: /Users/catiasantos/Documents/Code/LDS/UAb_LDS_Grupo10_ChalkPowder/FicheirosTeste/Livro1.csv
[INFO] importar_ficheiro() - Início da importação
[INFO] importar_ficheiro() - 5 registos carregados do CSV
[INFO] user_seleccionou_grafico() - Tipo de gráfico selecionado: Barras
[INFO] user_seleccionou_grafico() - Colunas disponíveis: ['Categoria', 'Valor']
[INFO] on_grafico_seleccionado() - Gráfico selecionado: Barras
[INFO] on_submeter_parametros() - Validação de parâmetros iniciada.
[INFO] on_submeter_parametros() - Parâmetros validados: x=Categoria, y=Valor, x_label=Categoria, y_label=Valor
[INFO] user_submeteu_parametros() - Parâmetros submetidos: x=Categoria, y=Valor, x_label='Categoria', y_label='Valor'
[INFO] gerar_grafico() - A gerar gráfico do tipo 'Barras'
[INFO] gerar_grafico() - Gráfico gerado com sucesso
[INFO] mostrar_grafico() - Interface de gráfico exibida.
[INFO] mostra_dlg_grava_grafico_evt - Utilizador solicitou guardar o gráfico
[INFO] on_guardar_grafico_click() - Botão 'Guardar Gráfico' clicado.
[INFO] on_importar_ficheiro_click() - Botão 'Importar Ficheiro' clicado.
[INFO] mostra_dlg_carregar_ficheiro() - Ficheiro selecionado: /Users/catiasantos/Documents/Code/LDS/UAb_LDS_Grupo10_ChalkPowder/FicheirosTeste/Livro1.csv
[INFO] importar_ficheiro() - Início da importação
[INFO] importar_ficheiro() - 5 registos carregados do CSV
```